Aula — Testes de Integração com Spring Boot e JUnit 5

Disciplina: Projeto, Implementação e Teste de Software

Duração sugerida: 2 aulas de 50 minutos

Pré-requisitos: Java, orientação a objetos, Maven, JUnit 5 e testes unitários.

1. Objetivos da aula

Ao final da aula, os alunos deverão conseguir:

1. Diferenciar testes unitários de testes de integração.
2. Identificar os componentes de uma aplicação Spring Boot.
3. Compreender o papel da injeção de dependências.
4. Utilizar `@SpringBootTest` para carregar o contexto da aplicação.
5. Criar testes de integração utilizando JUnit 5.
6. Testar a comunicação entre serviços, repositórios e banco de dados.
7. Compreender a importância do isolamento dos dados entre testes.

Parte 1 — Recuperando os conceitos anteriores

2. O que já estudamos?

Nos exemplos anteriores, utilizamos duas classes:

```
public class EstoqueService {

    private int quantidade;

    public EstoqueService(int quantidade) {
        this.quantidade = quantidade;
    }

    public boolean temEstoque(int quantidadeSolicitada) {
        return quantidade >= quantidadeSolicitada;
    }

    public void retirar(int quantidade) {
        this.quantidade -= quantidade;
    }

    public int getQuantidade() {
        return quantidade;
    }
}
```

E:

```
public class PedidoService {

    private final EstoqueService estoque;

    public PedidoService(EstoqueService estoque) {
        this.estoque = estoque;
    }

    public boolean realizarPedido(int quantidade) {
        if (!estoque.temEstoque(quantidade)) {
            return false;
        }
        estoque.retirar(quantidade);
        return true;
    }
}
```

Pergunta para a turma

Quando executamos:

```
EstoqueService estoque = new EstoqueService(10);
PedidoService pedido = new PedidoService(estoque);
pedido.realizarPedido(3);
```

Quantas classes estão participando da operação? Duas. A operação depende da colaboração entre `PedidoService` e `EstoqueService`.

3. Teste unitário versus teste de integração

Característica	Teste unitário	Teste de integração
Objetivo	Verificar uma unidade isoladamente	Verificar a colaboração entre componentes
Dependências	Geralmente isoladas	Componentes reais envolvidos no teste
Banco de dados	Normalmente substituído	Pode ser utilizado
Velocidade	Geralmente maior	Geralmente menor
Exemplo	Testar cálculo de desconto	Testar serviço que salva um pedido

Importante: um teste de integração não precisa envolver banco de dados. Entretanto, integrar a aplicação com a camada de persistência é um exemplo útil.

Parte 2 — O que muda com Spring Boot?

4. Uma aplicação organizada em camadas

Considere um sistema de gerenciamento de produtos.



- Controller: recebe requisições HTTP e produz respostas.
- Service: implementa regras de negócio.
- Repository: realiza operações de persistência.
- Banco de dados: armazena os registros.

Problema: como verificar se esses componentes funcionam corretamente em conjunto? Por meio de testes de integração.

5. Injeção de dependências

Em Java convencional, poderíamos criar os objetos manualmente:

```
ProdutoRepository repository = new ProdutoRepository();
ProdutoService service = new ProdutoService(repository);
```

No Spring Boot, os componentes podem ser gerenciados pelo contêiner do Spring:

```
@Service
public class ProdutoService {
    private final ProdutoRepository repository;

    public ProdutoService(ProdutoRepository repository) {
        this.repository = repository;
    }
}
```

`@Service` identifica a classe como componente de serviço. O Spring fornece a dependência apropriada.

Parte 3 — Construindo uma aplicação de exemplo

6. Nosso problema

Desenvolver uma aplicação que permita cadastrar e consultar produtos.

Regras de negócio:

- RN01: um produto deve possuir um nome.
- RN02: o preço deve ser maior que zero.
- RN03: um produto cadastrado deve ser armazenado no banco de dados.
- RN04: um produto cadastrado deve poder ser recuperado por identificador.

Tecnologias: Java 17+, Spring Boot 3.x, Maven, Spring Data JPA, H2, JUnit 5 e Spring Boot Test.

7. Criando a entidade Produto

```
package br.edu.exemplo.model;

import jakarta.persistence.*;

@Entity
public class Produto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nome;
    private double preco;

    protected Produto() {}

    public Produto(String nome, double preco) {
        this.nome = nome;
        this.preco = preco;
    }
```

```
    public Long getId() { return id; }
    public String getNome() { return nome; }
    public double getPreco() { return preco; }
}
```

- `@Entity`: entidade persistente.
- `@Id`: chave primária.
- `@GeneratedValue`: geração do identificador.

8. Criando o Repository

```
package br.edu.exemplo.repository;

import br.edu.exemplo.model.Produto;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ProdutoRepository extends JpaRepository<Produto, Long> {
}
```

O Spring Data JPA fornece operações como `save()`, `findById()`, `findAll()` e `deleteById()`.

9. Criando o Service

```
package br.edu.exemplo.service;

import br.edu.exemplo.model.Produto;
import br.edu.exemplo.repository.ProdutoRepository;
import org.springframework.stereotype.Service;

@Service
public class ProdutoService {

    private final ProdutoRepository repository;

    public ProdutoService(ProdutoRepository repository) {
        this.repository = repository;
    }

    public Produto cadastrar(String nome, double preco) {
        if (nome == null || nome.isBlank()) {
            throw new IllegalArgumentException("Nome obrigatório");
        }
        if (preco <= 0) {
```

```
            throw new IllegalArgumentException("Preço deve ser positivo");
        }
        Produto produto = new Produto(nome, preco);
        return repository.save(produto);
    }

    public Produto buscar(Long id) {
        return repository.findById(id)
            .orElseThrow(() -> new IllegalArgumentException("Produto não encontrado"));
    }
}
```

O que queremos testar? Recebimento dos dados, validação das regras, chamada ao repositório, persistência e recuperação do registro.

Parte 4 — Primeiro teste de integração

10. A anotação `@SpringBootTest`

```
@SpringBootTest
class ProdutoServiceIntegrationTest {
}
```

A anotação carrega o contexto da aplicação para os testes. Não inicia obrigatoriamente um servidor HTTP; isso depende da configuração.

11. Nosso primeiro teste

```
package br.edu.exemplo;

import br.edu.exemplo.model.Produto;
import br.edu.exemplo.repository.ProdutoRepository;
import br.edu.exemplo.service.ProdutoService;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.transaction.annotation.Transactional;

import static org.junit.jupiter.api.Assertions.*;

@SpringBootTest
@Transactional
class ProdutoServiceIntegrationTest {

    @Autowired
    private ProdutoService service;

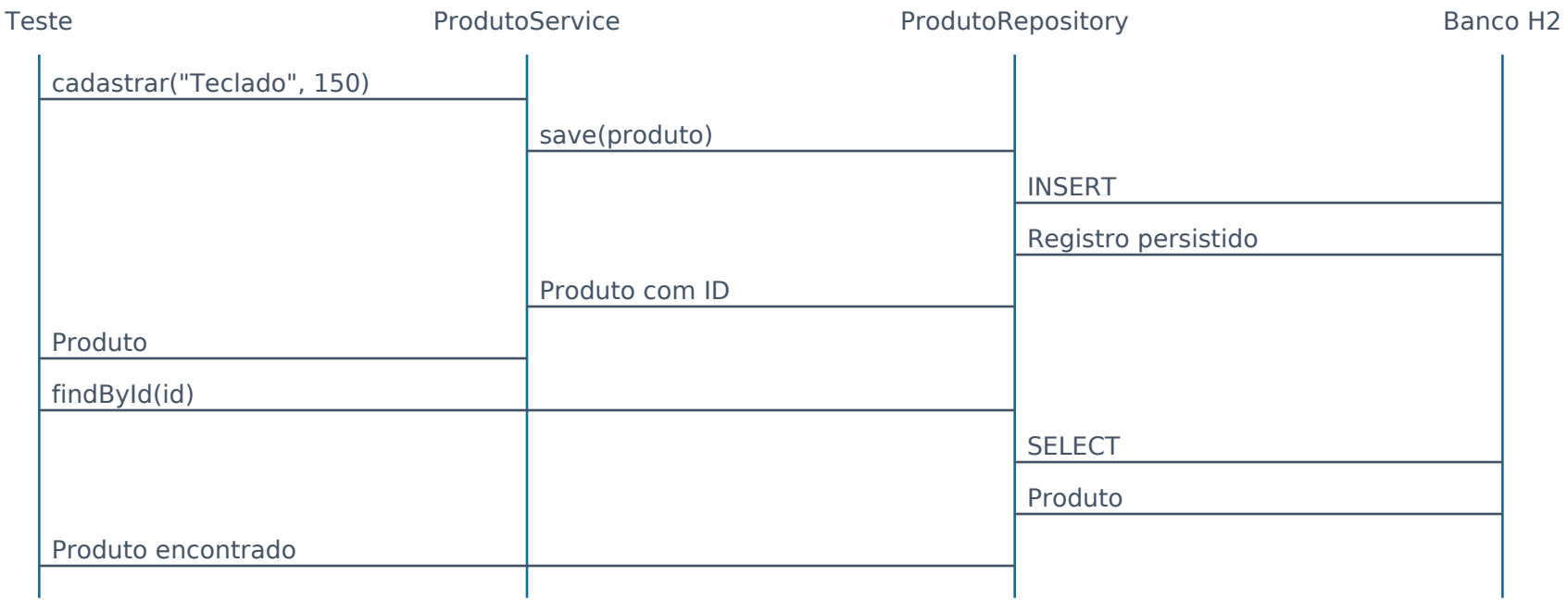
    @Autowired
```

```
    private ProdutoRepository repository;

    @Test
    void deveCadastrarProdutoNoBanco() {
        // Arrange
        String nome = "Teclado";
        double preco = 150.0;

        // Act
        Produto produto = service.cadastrar(nome, preco);

        // Assert
        assertNotNull(produto.getId());
        Produto encontrado = repository.findById(produto.getId()).orElseThrow();
        assertEquals("Teclado", encontrado.getNome());
        assertEquals(150.0, encontrado.getPreco());
    }
}
```



O teste verifica a colaboração real entre serviço, repositório e persistência. Nota: o diagrama é lógico; o momento de execução do SQL depende do JPA e do `flush()`.

Parte 5 — Verificando regras de negócio

12. Testando um produto inválido

Regra: não permitir preço igual ou inferior a zero.

```
@Test
void naoDeveCadastrarProdutoComPrecoNegativo() {
    long quantidadeAntes = repository.count();

    assertThrows(
        IllegalArgumentException.class,
        () -> service.cadastrar("Mouse", -50)
    );

    assertEquals(quantidadeAntes, repository.count());
}
```

O teste verifica rejeição do cadastro e ausência de novos registros. Como a exceção ocorre antes de acessar o repositório, o comportamento exercitado é principalmente a validação do serviço, apesar do ambiente de integração.

Parte 6 — Isolamento dos testes

13. O problema dos dados persistidos

```
Teste A → cadastra produto → banco contém 1 produto
Teste B → espera banco vazio → falha
```

Os testes não devem depender da ordem de execução. Uma estratégia comum é:

```
@Transactional
```

Nos testes transacionais gerenciados pelo Spring, normalmente ocorre rollback ao final de cada teste.

```
Início do teste → transação → operações → verificações → rollback
```

Limitação: operações em outras threads ou transações independentes podem não participar da transação do teste.

Parte 7 — Testando a camada HTTP

14. E se quisermos testar o Controller?

Até aqui:

```
ProdutoService → ProdutoRepository → Banco H2
```

Agora:

```
Requisição HTTP → ProdutoController → ProdutoService → ProdutoRepository → Banco H2
```

O `MockMvc` simula requisições HTTP sem iniciar um servidor HTTP real.

15. Criando o Controller

```
package br.edu.exemplo.controller;

import br.edu.exemplo.model.Produto;
import br.edu.exemplo.service.ProdutoService;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/produtos")
public class ProdutoController {

    private final ProdutoService service;

    public ProdutoController(ProdutoService service) {
        this.service = service;
    }

    @PostMapping
    public Produto cadastrar(@RequestBody ProdutoRequest request) {
        return service.cadastrar(request.nome(), request.preco());
    }
```

```
    @GetMapping("/{id}")
    public Produto buscar(@PathVariable Long id) {
        return service.buscar(id);
    }
}
```

```
package br.edu.exemplo.controller;

public record ProdutoRequest(String nome, double preco) {}
```

16. Teste de integração com MockMvc

```
package br.edu.exemplo;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.transaction.annotation.Transactional;

import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@SpringBootTest
@AutoConfigureMockMvc
@Transactional
class ProdutoControllerIntegrationTest {

    @Autowired
    private MockMvc mockMvc;
```

```
    @Test
    void deveCadastrarProdutoPelaApi() throws Exception {
        mockMvc.perform(
                post("/produtos")
                    .contentType(MediaType.APPLICATION_JSON)
                    .content("""
                        {
                            "nome": "Teclado",
                            "preco": 150.0
                        }
                    """)
            )
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.nome").value("Teclado"))
            .andExpect(jsonPath("$.preco").value(150.0))
            .andExpect(jsonPath("$.id").isNumber());
    }
}
```

Verificações: requisição, desserialização JSON, Controller, Service, Repository, persistência e resposta HTTP.

Parte 8 — Estratégias de teste no Spring Boot

17. Nem todo teste precisa carregar a aplicação inteira

Estratégia	Objetivo
JUnit 5 sem Spring	Testar classes isoladamente
`@DataJpaTest`	Testar a camada de persistência
`@WebMvcTest`	Testar a camada web isoladamente
`@SpringBootTest`	Testar a integração com o contexto completo
`@SpringBootTest` + `MockMvc`	Testar fluxos HTTP com componentes reais

Exemplo: `@DataJpaTest`

```
@DataJpaTest
class ProdutoRepositoryTest {

    @Autowired
    private ProdutoRepository repository;

    @Test
    void devePersistirProduto() {
        Produto produto = new Produto("Mouse", 80);
        Produto salvo = repository.save(produto);
        assertNotNull(salvo.getId());
    }
}
```

Parte 9 — Atividade prática

18. Evolução do sistema de produtos

Requisito 1 — Buscar produto existente: cadastrar e buscar pelo identificador, verificando os dados.

Requisito 2 — Buscar produto inexistente: verificar a exceção prevista para identificador inexistente.

Requisito 3 — Não permitir nome vazio: rejeitar nome vazio ou só com espaços e confirmar que não houve persistência.

Requisito 4 — Atualizar preço: cadastrar, atualizar e consultar novamente para verificar o novo valor.

Requisito 5 — Remover produto: cadastrar, remover e verificar que o identificador não é mais encontrado.

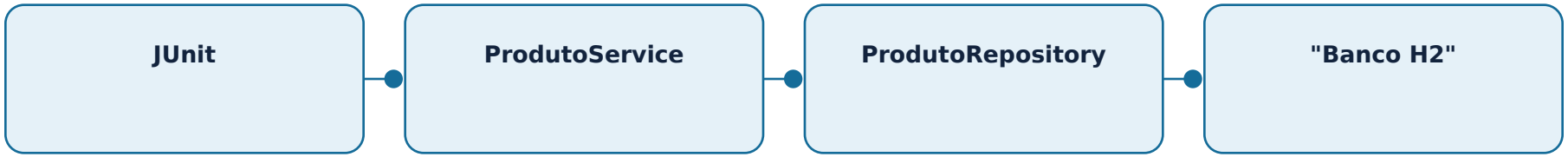
Parte 10 — Fechamento

19. Qual é a principal diferença?

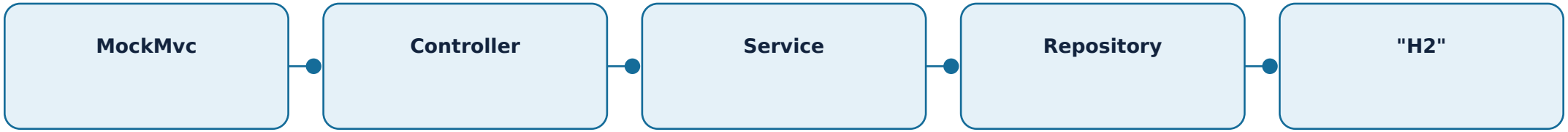
Teste unitário:



Teste de integração:



Teste de integração envolvendo HTTP:



20. Perguntas para discussão

1. Se um método funciona isoladamente, ele necessariamente funciona integrado? Não. Podem existir problemas de comunicação, configuração, persistência e dados.
2. Todo teste com `@SpringBootTest` é um bom teste de integração? Não. Depende das interações efetivamente exercitadas.
3. Por que não usar apenas testes de integração? Testes unitários tendem a ser mais rápidos e ajudam a localizar defeitos.
4. Por que não usar apenas testes unitários? Eles não garantem que componentes e infraestrutura reais funcionem juntos.

Sequência didática recomendada

Etapa	Conteúdo	Prática
1	Testes unitários	Conta Bancária
2	TDD	Estoque e Carrinho
3	Integração entre objetos Java	PedidoService + EstoqueService
4	Introdução ao Spring Boot	Controller, Service e Repository
5	Integração com persistência	ProdutoService + Repository + H2
6	Integração HTTP	MockMvc
7	Exercício autônomo	Evolução do cadastro de produtos

Resultado esperado: compreender que testes unitários verificam unidades isoladas, enquanto testes de integração verificam componentes e dependências reais funcionando em conjunto.

Observação de implementação: exemplos destinados ao Spring Boot 3.x, com `jakarta.persistence`. Para executar, são necessárias as dependências `spring-boot-starter-web`, `spring-boot-starter-data-jpa`, `h2`

e `spring-boot-starter-test`, além da classe principal com `@SpringBootApplication`.